

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Магнитогорский государственный технический университет им. Г. И. Носова»
Многопрофильный колледж

Отчет
по учебной практике
по специальности **09.02.07 Информационные системы и программирование (базовой**
подготовки)

ПМ. 02. Осуществление интеграции программных модулей

Студентов гр. ИСПП-21-2 Дувакин А.А,
Микрюкова А.А.
(И.О. Фамилия)

Руководитель практики от МпК

(И.О. Фамилия)

Магнитогорск, 2024

ВНУТРЕННЯЯ ОПИСЬ

документов, находящихся в отчете

Студентов гр. ИСП-21-2 Дувакин А.А., Микрюкова А.А.

(И.О. Фамилия)

№п/п	Наименование документа	Стр.
1	Задание на практику	3
2	Аттестационный лист по практике	1
3	Отчет о выполнении заданий по практике	16
4	Приложение А	8

ЗАДАНИЕ

на учебную практику

Студентов гр. ИСПП-21-2 Дувакин А.А., Микрюкова А.А.

(И.О. Фамилия)

09.02.07 Информационные системы и программирование (базовой подготовки)

ПМ. 02. Осуществление интеграции программных модулей

Учебная практика

Цели практики:

1. Получение практического опыта:

2.1 Разработка и оформление требований к программным модулям по предложенной документации.

2.2 Разработка тестовых наборов (пакетов) для программного модуля.

2.3 Разработка тестовых сценариев программного средства.

2.4 Инспектирование разработанных программных модулей на предмет соответствия стандартам кодирования.

2.5 Интеграция модулей в программное обеспечение.

2.6 . Отладка программных модулей.

Формирование профессиональных компетенций (ПК)

Код и наименование формируемых компетенций	Виды и объем производственных работ, выполняемых в период практики в рамках формируемых компетенций
ПК2.1 Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент	Выработка требований к программному обеспечению и программному модулю. Построение структуры программного продукта. Проектирование программного продукта.
ПК2.2 Выполнять интеграцию модулей в программное обеспечение	Внешнее проектирование (разработка внешней спецификации, разработка тестов)
ПК2.3 Выполнять отладку программного модуля с использованием специализированных программных средств	Внутреннее проектирование (разработка схем проекта)
ПК2.4 Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения	Разработка документа «Техническое задание» (разработка и оформление документа, согласование документа с заказчиком и руководителем, корректировка документа)
ПК2.5 Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования	Структурное проектирование программных модулей Проектирование программного обеспечения с использованием программных средств Приведение программных средств в соответствие со стандартами Разработка требований к программному проекту Определение необходимых ресурсов для создания программного продукта Написание программного кода программного обеспечения. Интеграция проектируемых модулей в компьютерную систему Оценка надежности интегрируемых модулей

	Отладка программных продуктов Разработка тестовых наборов для заданного программного продукта Проведения верификации программного продукта Разработка пользовательской документации Разработка руководства по применению Постановка задачи. Выбор математической модели. Выбор и обоснование численного метода. Разработка программы. Решение модельных примеров. Сопоставление с теорией. Решение прикладной задачи.
--	--

Формирование общих компетенций (ОК)

ОК 1. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам.

ОК 2 . Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности.

ОК 3. Планировать и реализовывать собственное профессиональное и личностное развитие.

ОК 4. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами.

ОК 5. Осуществлять устную и письменную коммуникацию на государственном языке с учетом особенностей социального и культурного контекста.

ОК 6. Проявлять гражданско-патриотическую позицию, демонстрировать осознанное поведение на основе традиционных общечеловеческих ценностей.

ОК 7. Содействовать сохранению окружающей среды, ресурсосбережению, эффективно действовать в чрезвычайных ситуациях.

ОК 8. Использовать средства физической культуры для сохранения и укрепления здоровья в процессе профессиональной деятельности и поддержания необходимого уровня физической подготовленности.

ОК 9. Использовать информационные технологии в профессиональной деятельности.

ОК10. Пользоваться профессиональной документацией на государственном и иностранном языках.

ОК11. Планировать предпринимательскую деятельность в профессиональной сфере.

Место практики МпК, полигон учебных баз практик

Задание на практику

№ п/п	Содержание работ на практике	Примерные сроки выполнения (час)
1.	Разработка документа «Техническое задание» (разработка и оформление документа, согласование документа с заказчиком и руководителем, корректировка документа)	6
2.	Постановка задачи.	6
3.	Выбор математической модели.	6
4.	Проектирование программного обеспечения с использованием программных средств	12
5.	Внутреннее проектирование (разработка схем проекта)	18
6.	Разработка программы.	12
7.	Написание программного кода программного обеспечения	12
8.	Разработка и модификация модулей программного обеспечения в соответствии с требованиями заказчика	18
9.	Отладка программных модулей и выявление ошибок программного обеспечения	12
10.	Разработка руководства по применению	6

ПМ. 02. Осуществление интеграции программных модулей

Руководитель практики от МпК

(подпись)

И.О. Фамилия

«06» май 2024г.

АТТЕСТАЦИОННЫЙ ЛИСТ ПО УЧЕБНОЙ ПРАКТИКЕ

Дуванкин А.А., Микрюкова А.А.

(И.О.Фамилия)

обучающихся на 3 курсе специальности 09.02.07 Информационные системы и программирование

(шифр и наименование специальности)

успешно прошли учебную практику по профессиональному модулю:

ПМ. 02. Осуществление интеграции программных модулей

в объеме 72 часов с «06» мая 2024 г. по «18» мая 2024 г. в организации

ФГБОУ ВО «Магнитогорский государственный технический университет»,

многопрофильный колледж

(наименование организации, юридический адрес)

Виды и качество выполнения работ

Код формируемых компетенций	Виды и объем работ, выполненных обучающимися во время практики в рамках формируемых компетенций	Оценка зачтено/ не зачтено
ПК2.1 Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент ПК2.2 Выполнять интеграцию модулей в программное обеспечение ПК2.3 Выполнять отладку программного модуля с использованием специализированных программных средств ПК2.4 Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения ПК2.5 Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования	Разработка документа «Техническое задание» (разработка и оформление документа, согласование документа с заказчиком и руководителем, корректировка документа)	
	Постановка задачи.	
	Выбор математической модели.	
	Проектирование программного обеспечения с использованием программных средств	
	Внутреннее проектирование (разработка схем проекта)	
	Разработка программы.	
	Написание программного кода программного обеспечения	
	Разработка и модификация модулей программного обеспечения в соответствии с требованиями заказчика	
	Отладка программных модулей и выявление ошибок программного обеспечения	
	Разработка руководства по применению	

Руководитель

практики

от

многопрофильного

колледжа

Дата «06» мая 2024г.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 ТРАНСПОРТНАЯ ЗАДАЧА

Математическое моделирование играет большую роль в решении различных экономических проблем, позволяя определить цели и типы их решения, обеспечивая структуру для целостного анализа. С помощью количественных моделей возможно более подробное изучение полученных данных, поэтому экономико-математическое моделирование является неотъемлемой частью любого исследования в области экономики. Ввиду сложности экономики для ее модельного описания используются различные подходы, одним из которых является линейное программирование.

Частью линейного программирования являются транспортные задачи, которые играют особую роль в уменьшении транспортных издержек предприятия. Это является актуальным вопросом в условиях рыночной экономики, когда любые затраты должны быть минимизированы, ведь тогда издержки покрываются меньшей частью прибыли, а также позволяют снизить себестоимость продукции на рынке, что делает предприятие более конкурентоспособным.

Транспортная задача – задача об оптимальном плане перевозок продукта из пункта наличия в пункт потребления. Их целью является доставка продукции в определённое время и место при минимальных совокупных затратах трудовых, материальных и финансовых ресурсов.

Она считается достигнутой, если нужный товар требуемого качества и в необходимом количестве доставляется в нужное время и в нужное место с минимальными затратами.

Выделяют два типа транспортных задач: по критерию стоимости – план перевозок является оптимальным, если достигается минимум затрат на его реализацию; по критерию времени – план перевозок оптимален, если на него затрачивается минимальное количество времени.

Для решения транспортных задач разработан специальный метод, имеющий следующие этапы:

- нахождение исходного опорного решения;
- проверка этого решения на оптимальность;
- переход от одного опорного решения к другому.

Существуют следующие методы решения транспортных задач:

1. Метод северо-западного угла заключается в том, что на каждом этапе левая верхняя (т.е. северо-западная) клетка заполняется максимальным числом. Заполнение продолжается до тех пор, пока на одном из шагов не исчерпаются запасы и не удовлетворятся все потребности.

2. Метод потенциалов решения. Определяют систему из $m+1$ линейных уравнений с $(m+n)$ неизвестными, имеющую бесчисленное множество решений; для её определённости одному неизвестному присваивают произвольное значение (обычно альфа равно 0), тогда все остальные неизвестные определяются однозначно.

3). Метод минимального элемента заключается в заполнении на каждом шаге таблицы той клетки, которой соответствует наименьшее значение, а в случае наличия нескольких одинаковых тарифов заполняется любой из них.

4. Метод аппроксимации Фогеля – более трудоёмкий, но начальный план перевозок, построенный с его помощью, является наиболее приближенным к оптимальному. При решении задачи данным методом по всем строкам и столбцам таблицы находится разность между минимальными тарифами (строка или столбец с наибольшей разницей является предпочтительным). В пределах выбранной строки (столбца) находится ячейка с наименьшим тарифом, на которую записывают отгрузку. Строки поставщиков, которые полностью исчерпали возможности по отгрузке, и столбцы потребителей, потребности которых удовлетворены, вычёркиваются.

1.2 Дельта-метод решения транспортной задачи

Дельта-метод (или метод северо-западного угла и потенциалов) - это один из методов решения транспортной задачи, который комбинирует идеи метода северо-западного угла и метода потенциалов для эффективного нахождения оптимального решения. Он основан на идее поочерёдного заполнения ячеек матрицы перевозок и вычисления потенциалов для определения оптимального плана.

Шаги метода Дельта:

1. Преобразуем таблицу C_{ij} в таблицу приращений ΔC_{ij} , выбирая в каждом столбце наименьшую стоимость и вычитая её из всех стоимостей столбца. Значения записываем под соответствующими значениями.

2. Таблицу преобразуем в таблицу $\overline{\Delta C_{ij}}$, выбирая в каждой строке наименьшее приращение и вычитая его из всех приращений строки; результаты записываем под значениями. Если в какой-нибудь строке уже имеется уже нулевое приращение после первого преобразования, то в этой строке приращения оставляем без изменения и преобразуем приращения строк, не содержащих нулевых приращений.

3. Просматриваем столбцы, содержащие одно нулевое приращение, и в клетки, содержащие его, записываем потребности b_j , не обращая внимания на величину запасов поставщиков. Затем просматриваем столбцы, содержащие два нулевых приращения, и

клетки, содержащие их, заполняем, учитывая ранее произведённое закрепление и запасы поставщиков. Затем переходим к столбцам, содержащим 3, 4, и так далее нулевых приращений. Процесс закрепления потребителей за поставщиками продолжается до тех пор, пока все объёмы потребителей не будут закреплены за поставщиками.

4. Подсчитываем для строк

$$\Delta a_i = a_i - \sum_{j=1}^n x_{ij};$$

$$i=1,2,\dots,m.$$
(1)

Если все $\Delta a_i = 0$, то построение плана закончено. Он же является оптимальным, т.к. все грузы перевозятся с наименьшими приращениями стоимостей. В общем случае получаем:

а) для одних строк, они называются нулевыми.

б) для других строк $\Delta a_i < 0$, они называются избыточными и отмечаются знаком "−".

в) для третьих, такие строки называются недостаточными и отмечаются знаком "+".

5. Отмечаем знаком "V" столбцы, имеющие занятые клетки в избыточных строках.

6. Для каждой недостаточной и нулевой строки сравниваем, стоящие в отмеченных столбцах, выбираем наименьшее и проставляем в последнюю графу таблицы.

7. В последней графе просматриваем недостаточных строк, выбираем наименьшее и сравниваем его с C_{i0j} для нулевых строк. При этом может быть два случая:

а) для каждой нулевой строки

$$\min \Delta \overline{C}_{ij} \leq \overline{C}_{i0j}$$
(2)

б) для некоторых нулевых строк

$$\min \Delta \overline{C}_{ij} > \overline{C}_{i0j}$$
(3)

8. Если выполняется условие (2), то производится непосредственное распределение потребности из избыточной строки в недостаточную клетку отмеченного столбца, которой соответствует $\min \Delta \overline{C}_{ij}$. Величина перераспределения находится по формуле (4).

$$\Delta x = \min(x_{ij}, |\Delta a_i|),$$
(4)

где x_{ij} - величина перевозки, стоящая в отмеченном столбце избыточной строки; Δa_i - величина разностей, стоящих в избыточной и недостаточной строках.

9. Если для некоторой нулевой строки выполняется условие (3), то перераспределение проверяем по цепочкам, идущим через эту нулевую строку из избыточной строки в

недостаточную. Для построения цепочки в этой нулевой строке в отмеченном столбце находим клетку, для которой $\min \Delta \overline{C_{ij}} < \Delta \overline{C_{i_0j}}$, и отмечаем ее знаком "+", в том же столбце находим занятую клетку, стоящую в избыточной строке, и отмечаем ее знаком "-" – это начало цепочки. Начиная движение по построенному звену цепочки от "-" к "+" попадаем в нулевую строку, затем передвигаемся по нулевой строке к занятой клетке и отмечаем её знаком "-", далее по столбцу переходим в клетку недостаточной строки и отмечаем её знаком "+". Цепочка построена, если матрица содержит большое число нулевых строк, то цепочки перераспределения могут проходить через несколько нулевых строк и их количество значительно возрастает, поэтому руководствуемся следующим правилом: При переходе из одной нулевой строки в другую определяем полученную сумму приращений и сравниваем её с минимумом приращений в выделенных столбцах данной строки. Если полученная сумма превышает этот минимум, то продолжение цепочки по данной строке не рассматриваем. Очевидно также, что, если сумма приращений, полученная при переходе в недостаточную строку, меньше, чем при переходе в нулевую строку.

10. Составляем для каждой цепочки алгебраическую сумму приращений $\sum_i \sum_j \Delta \overline{C_{ij}}$, считая их отрицательными, если они стоят в клетке, отмеченной знаком "-", и положительными, если клетка отмечена знаком "+". Полученную сумму сравниваем с:

а) если $\sum_i \sum_j \Delta \overline{C_{ij}} \geq \min \Delta \overline{C_{ij}}$ для всех построенных цепочек, то отбрасываем их и производим непосредственное перераспределение;

б) если $\sum_i \sum_j \Delta \overline{C_{ij}} < \min \Delta \overline{C_{ij}}$, то перераспределение по цепочке рассчитывается по формуле (5).

$$\Delta x = \min(x_{i_k j_p}, |\Delta a_{i_r}|), \quad (5)$$

где $x_{i_k j_p}$ – числа, указывающие на перевозки, которые стоят в клетках, отмеченных знаком "-", $1 \leq k \leq m, 1 \leq p \leq n$; Δa_{i_r} – разности, стоящие в избыточной и недостаточной строках, в которых начинается и заканчивается цепочка, $1 \leq r \leq m$. Следуя по цепочке, вычитаем величину перераспределения из чисел, помещённых в клетках, отмеченных знаком "-", и прибавляем к числам, которые стоят в клетках, помеченных знаком "+", на эту же величину изменяем $x_{i_k j_p}$. В результате получаем новое закрепление потребителей за поставщиками.

11. После перераспределения проверяем возможность исключения отмеченных столбцов. Столбцы исключаем из отмеченных в том случае, если занятая клетка избыточной строки превратилась в незанятую или избыточная строка превращается в нулевую. В этом случае следующую итерацию следует начать с пункта 6 алгоритма, если количество отмеченных столбцов осталось без изменения, то следующая итерация начинается с п.7 алгоритма.

12. Процесс перезакрепления продолжается до тех пор, пока все строки не превратятся в нулевые. Полученный план перевозок проверяют на оптимальность методом потенциалов.

2 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Алгоритм решения транспортной задачи

«Delta» - программа, которая позволяет решать транспортную задачу дельта-методом. Для начала работы необходим ввод количества магазинов и количества поставщиков, в полученную из данных показателей матрицу данные генерируются автоматически, но можно задать начальные данные самостоятельно (стоимости перевозок). Также как и ввод запасов и потребностей, для складов и магазинов соответственно.

После решения задачи, на экране появляется новая матрица с оптимальным планом перевозок, а также ответ (оптимальная стоимость перевозки).

Сама программа использует такие методы для решения задачи:

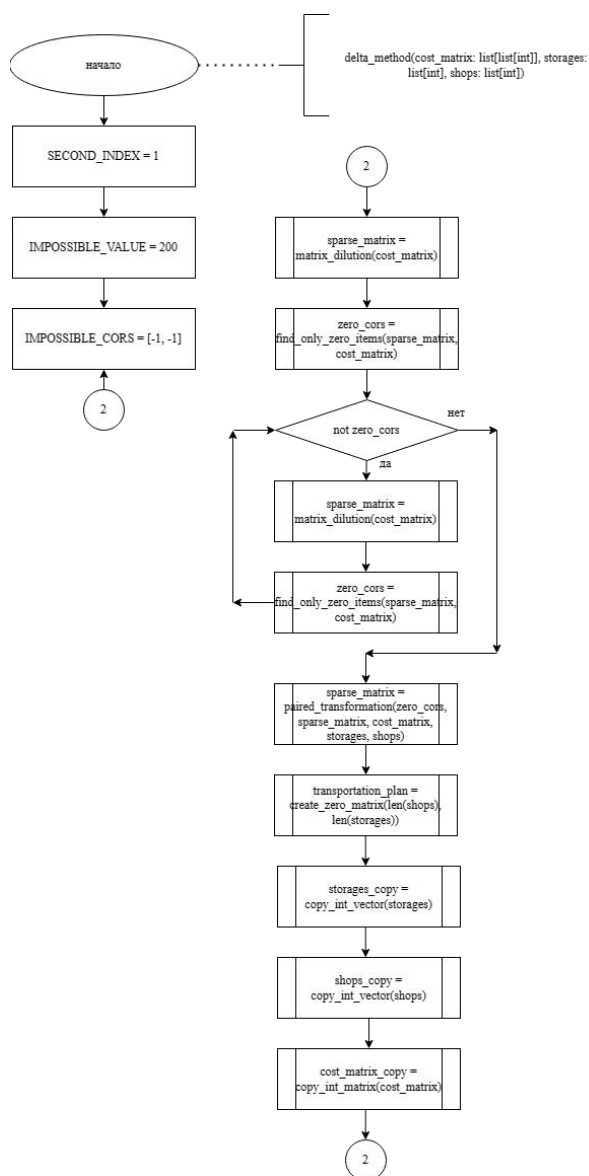


Рисунок 1 - delta_method

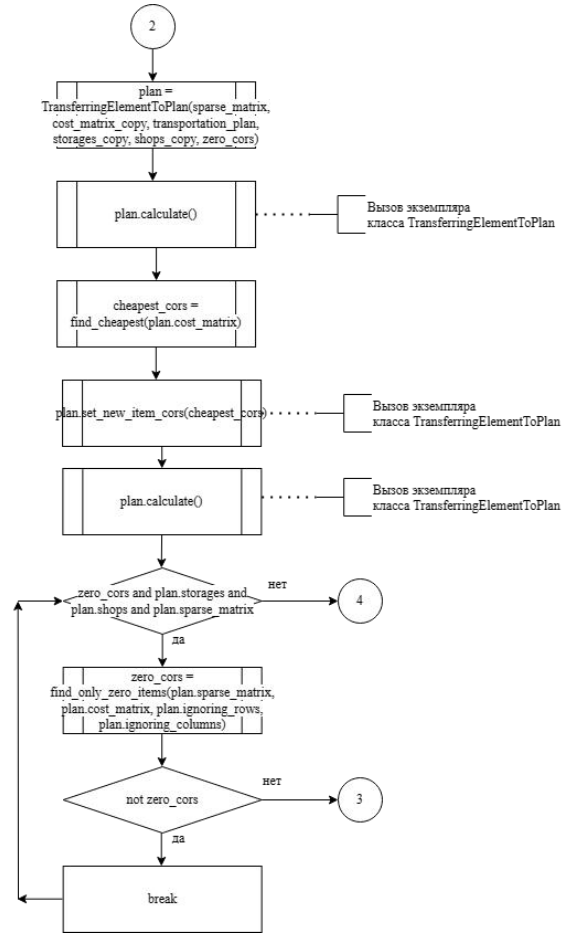


Рисунок 2 - delta_method

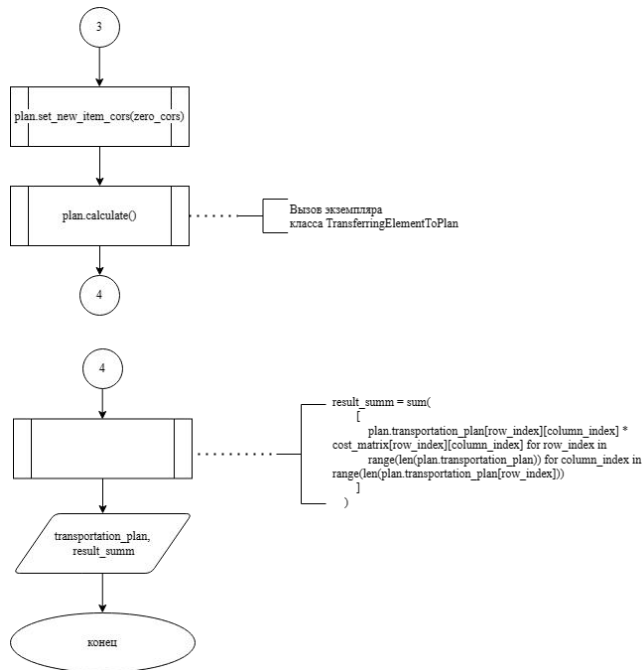


Рисунок 3 - delta_method

Это главная функция, которая координирует выполнение всех шагов алгоритма. Она принимает матрицу стоимости (cost_matrix), список складов (storages) и список магазинов (shops). Сначала она разреживает матрицу стоимости, затем ищет позиции с нулевыми элементами, иницирует процесс трансформации плана перевозок, ищет оптимальные элементы и пересчитывает план перевозок, пока это возможно. В конце возвращает план перевозок и общую стоимость перевозок (см. рисунок 1-3, листинг 1).

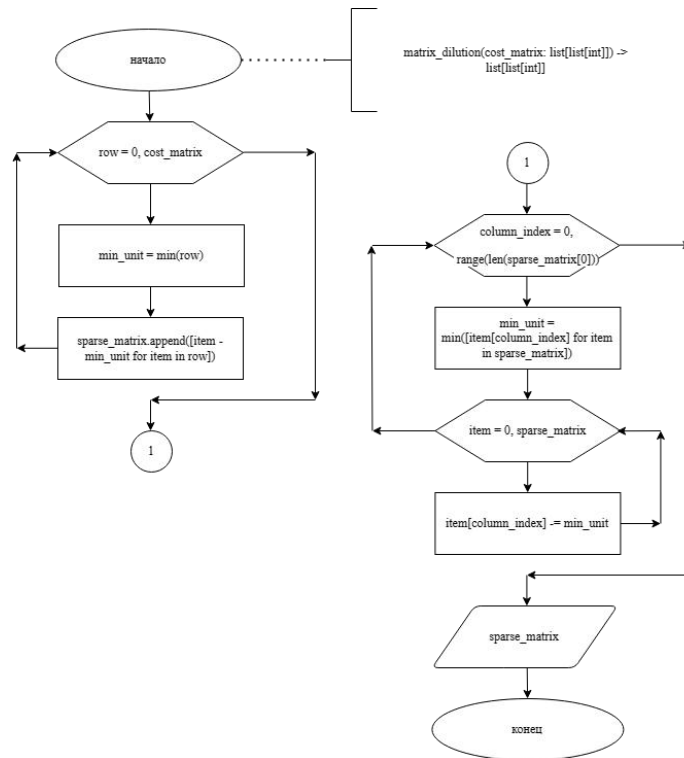


Рисунок 4 - matrix_dilution

Эта функция разреживает матрицу стоимости, вычитая из каждой строки и столбца минимальное значение соответствующей строки или столбца (см. рисунок 4, листинг 2).

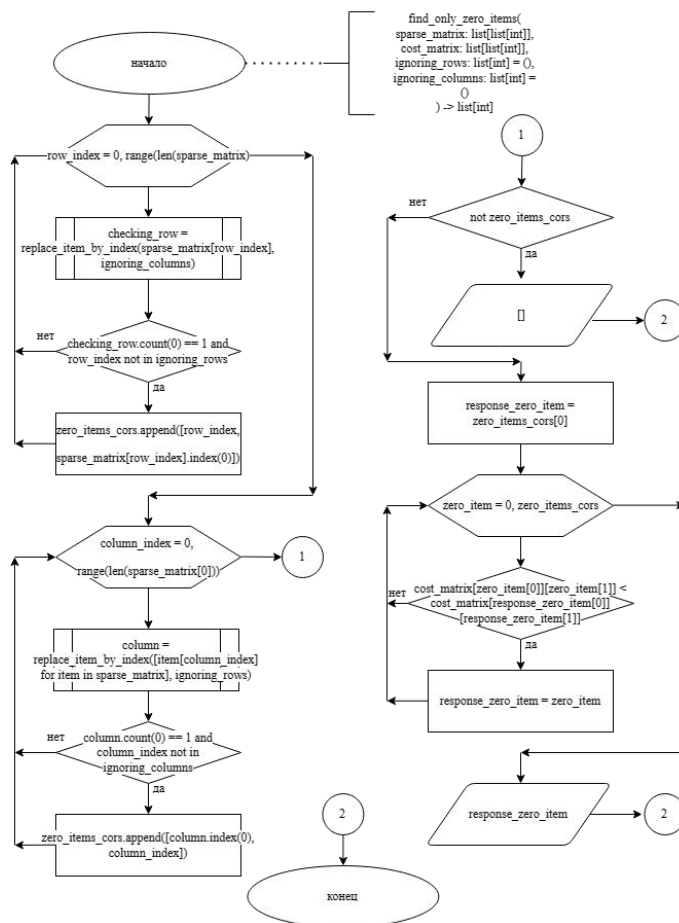


Рисунок 5 - find_only_zero_items

Она находит позиции в разреженной матрице, в которых есть только один нулевой элемент, и возвращает их (см. рисунок 5, листинг 3).

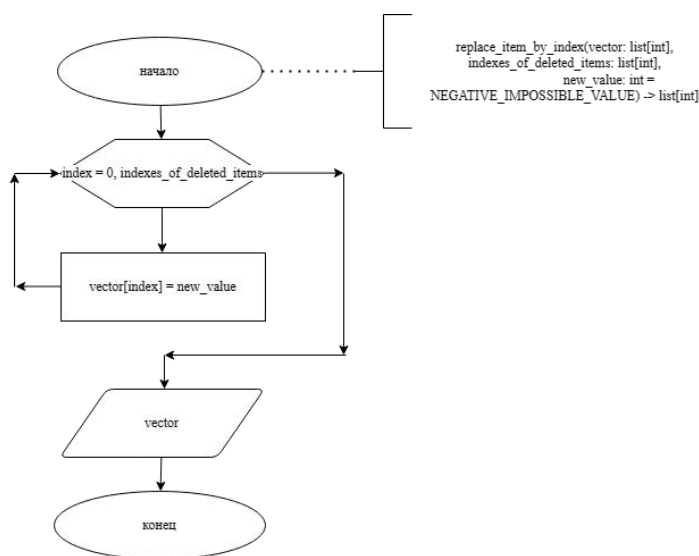


Рисунок 6 - replace_item_by_index

Эта функция заменяет элементы вектора по указанным индексам на новое значение (см. рисунок 6, листинг 4).

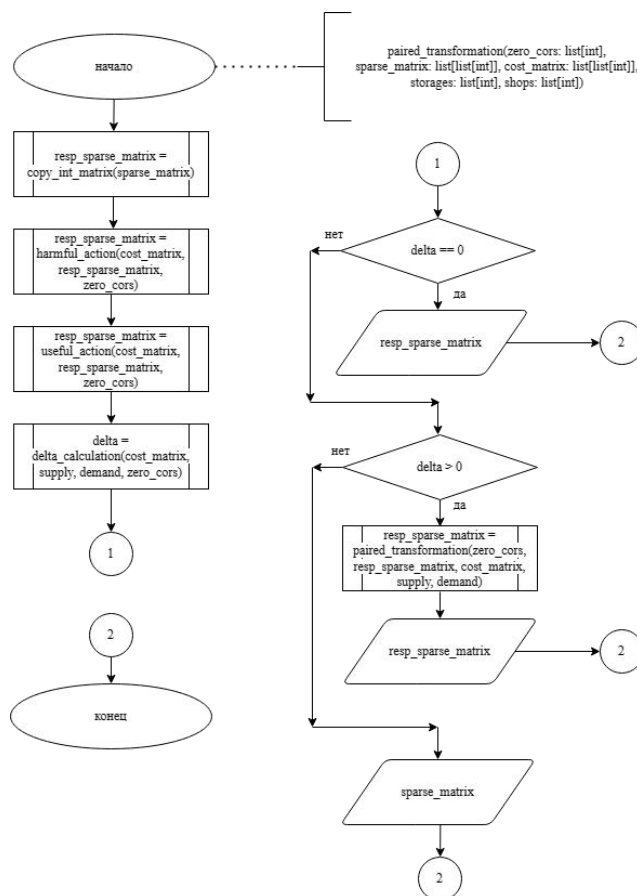


Рисунок 7 - paired_transformation

Эта функция реализует парную трансформацию нулевого элемента. Она изменяет матрицу в соответствии с алгоритмом, основанным на сравнении и расчете дельты (см. рисунок 7, листинг 5).

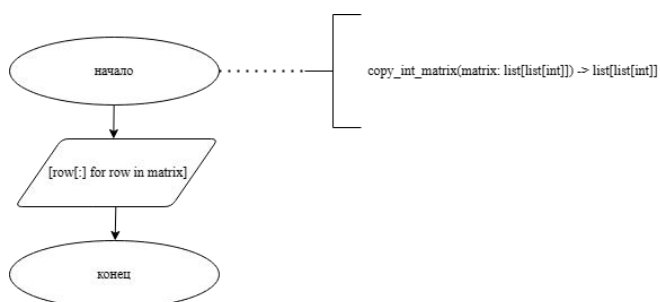


Рисунок 8 - copy_int_matrix

Функция копирует целочисленную матрицу (см. рисунок 8, листинг 6).

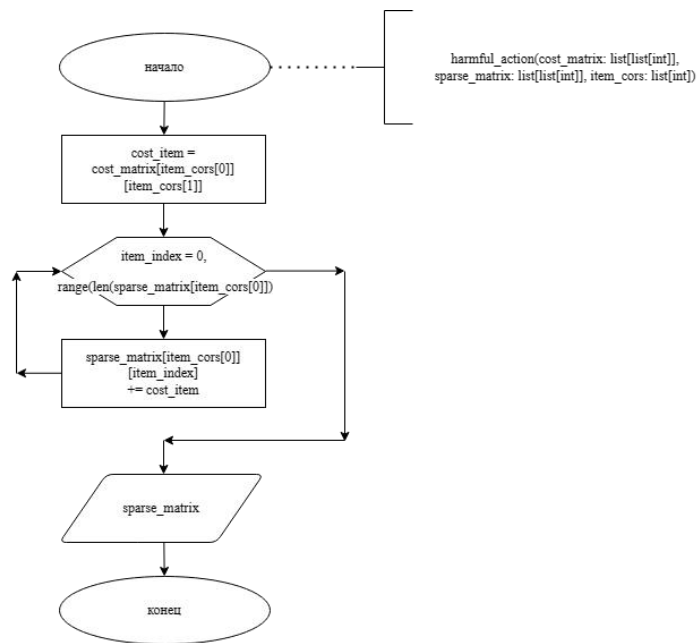


Рисунок 9 - harmful_action

Эта функция производит вредное действие для парной трансформации нулевого элемента (см. рисунок 9, листинг 7).

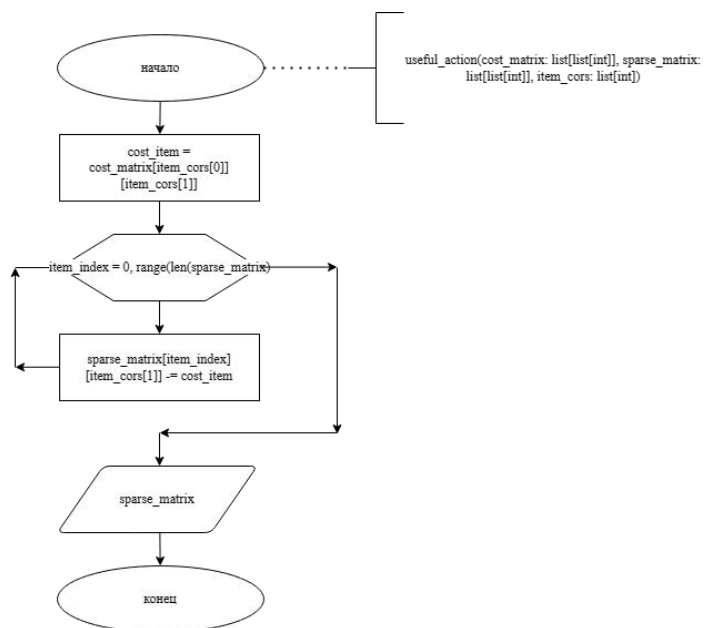


Рисунок 10 - useful_action

Эта функция производит полезное действие для парной трансформации нулевого элемента (см. рисунок 10, листинг 8).

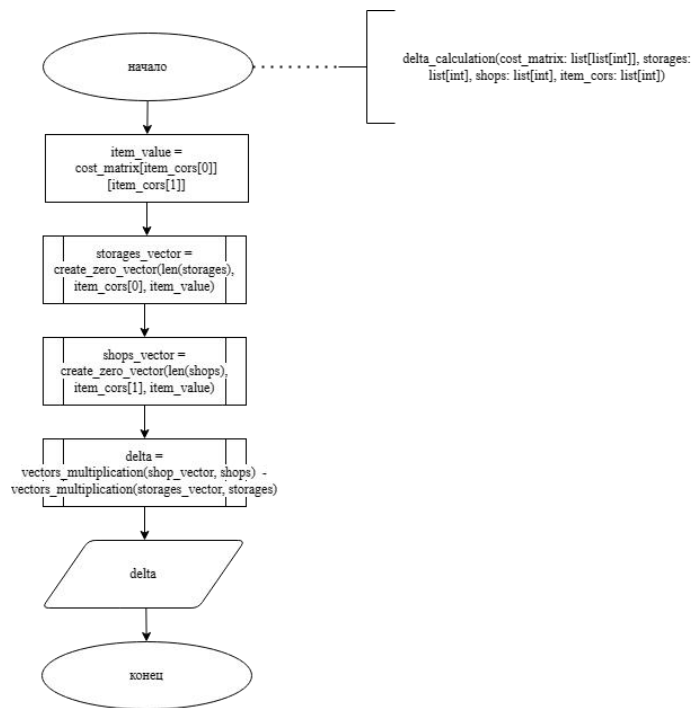


Рисунок 11 - delta_calculation

Эта функция рассчитывает дельту для парной трансформации нулевого элемента (см. рисунок 11, листинг 9).

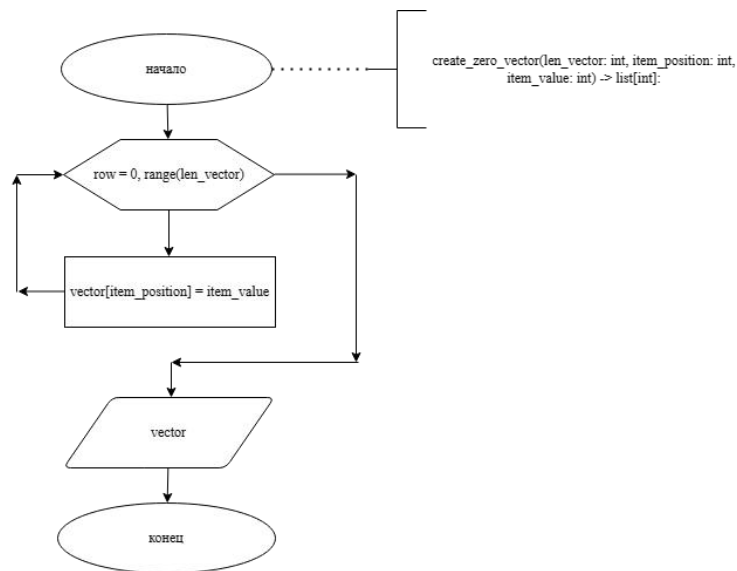


Рисунок 12 - create_zero_vector

Создаёт вектор с нулевыми значениями, за исключением одной позиции, где устанавливается указанное значение (см. рисунок 12, листинг 10).

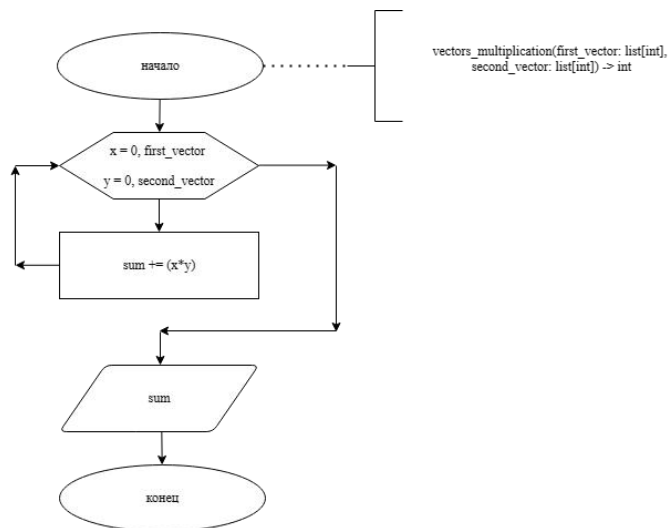


Рисунок 13 - vectors_multiplication

Умножает два вектора (см. рисунок 13, листинг 11).

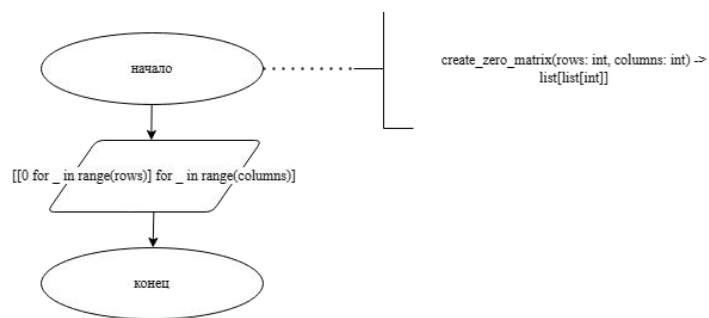


Рисунок 14 - create_zero_matrix

Создаёт матрицу с нулевыми значениями (см. рисунок 14, листинг 12).

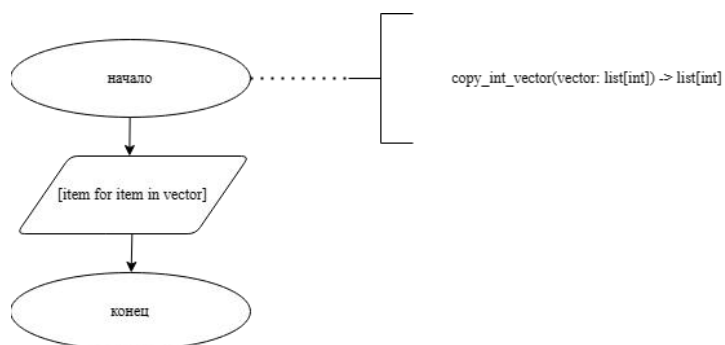


Рисунок 15 - copy_int_vector

Копирует целочисленный вектор (см. рисунок 15, листинг 13).

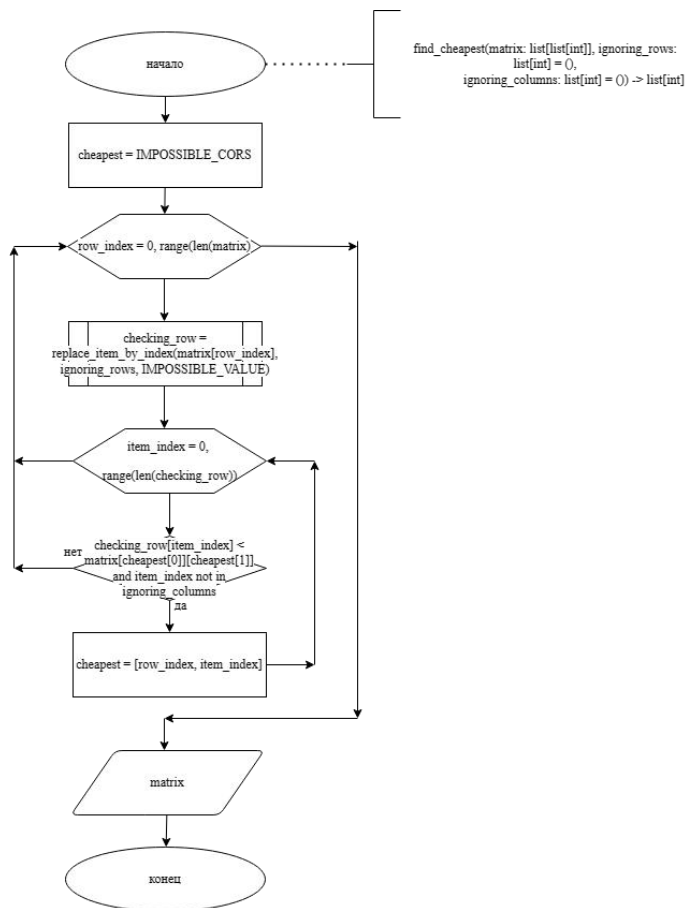


Рисунок 16 - find_cheapest

Находит позицию самого дешёвого элемента в матрице, игнорируя указанные строки и столбцы (см. рисунок 16, листинг 14).

2.2 Варианты решения транспортной задачи

Экран загрузки программного решения «Delta» представлен на рисунке 17.

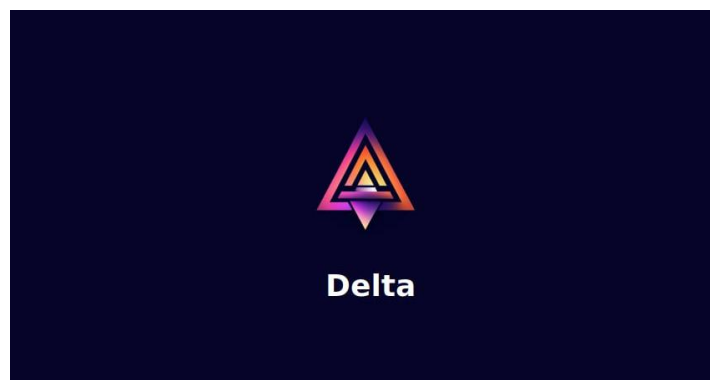


Рисунок 17 - Заставка

Экран решения транспортной задачи дельта-методом представлен на рисунках 18-19.

Транспортная задача - Дельта метод

Переключить тему

	Магазин 1	Магазин 2	Магазин 3	Магазин 4	Запасы
Поставщик 1	6	5	8	7	14
Поставщик 2	3	6	4	2	12
Поставщик 3	9	1	3	6	8
Потребности	10	14	6	4	

Магазины: 4 Поставщики: 3

Решить

	Магазин 1	Магазин 2	Магазин 3	Магазин 4	Запасы
Поставщик 1	8	6			14
Поставщик 2	2		6	4	12
Поставщик 3		8			8
Потребности	10	14	6	4	
Итог	124				

Рисунок 18 - Пример работы программы №1

Транспортная задача - Дельта метод

Переключить тему

	Магазин 1	Магазин 2	Магазин 3	Магазин 4	Запасы
Поставщик 1	6	5	8	7	14
Поставщик 2	3	6	4	2	12
Поставщик 3	9	1	3	6	8
Потребности	10	14	6	4	

Магазины: 4 Поставщики: 3

Решить

	Магазин 1	Магазин 2	Магазин 3	Магазин 4	Запасы
Поставщик 1	8	6			14
Поставщик 2	2		6	4	12
Поставщик 3		8			8
Потребности	10	14	6	4	
Итог	124				

Рисунок 19 - Пример работы программы №2

Пример ситуации, когда невозможно решить пример дельта-методом представлен на рисунках 20-21.

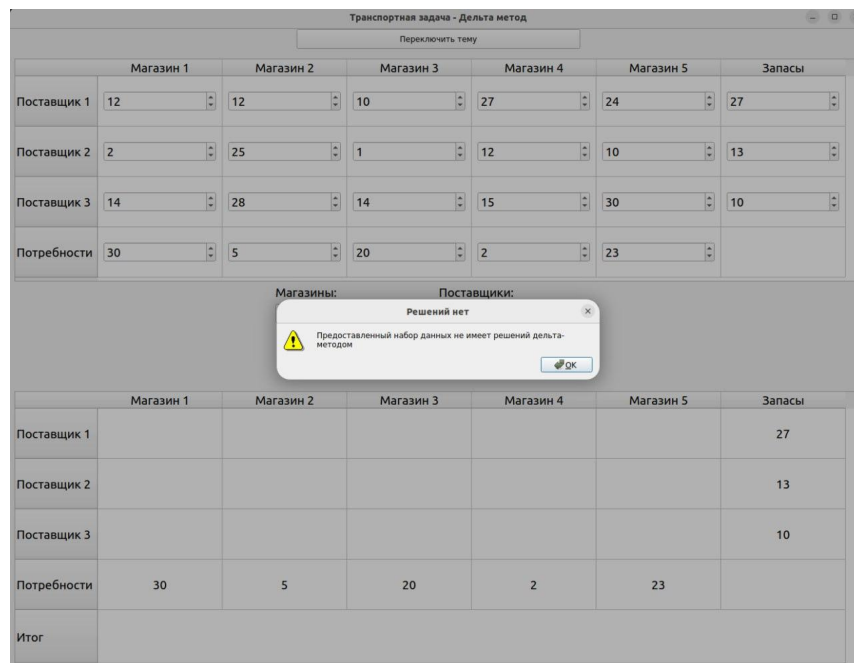


Рисунок 20 - Пример работы программы №3

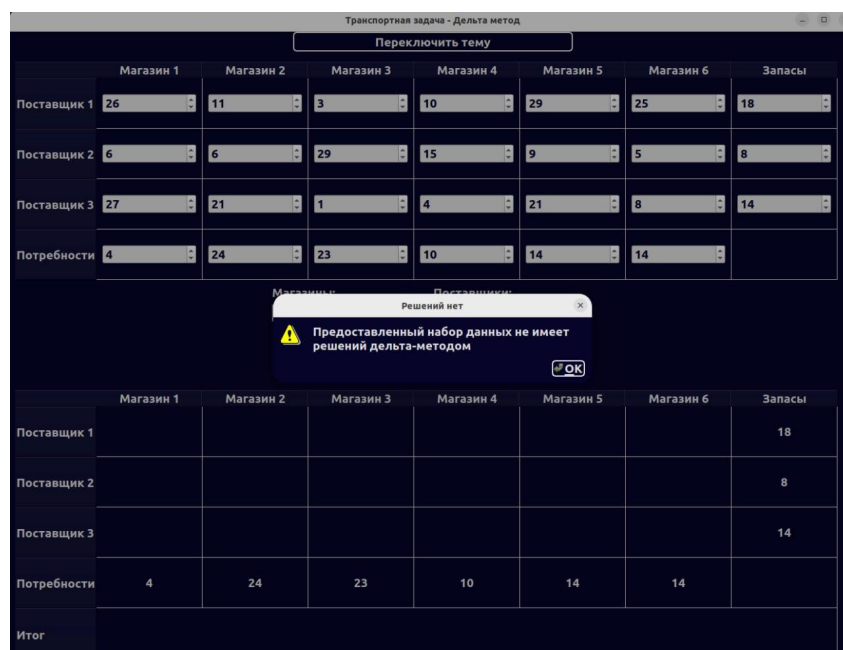


Рисунок 21 - Пример работы программы №4

ПРИЛОЖЕНИЕ А

Листинг 1 - delta_method

```
def delta_method(cost_matrix: list[list[int]], storages:
list[int], shops: list[int]):
    sparse_matrix = matrix_dilution(cost_matrix)
    zero_cors = find_only_zero_items(sparse_matrix, cost_matrix)

    while not zero_cors:
        sparse_matrix = matrix_dilution(cost_matrix)
        zero_cors = find_only_zero_items(sparse_matrix,
cost_matrix)

        sparse_matrix = paired_transformation(zero_cors, sparse_matrix,
cost_matrix, storages, shops)
        transportation_plan = create_zero_matrix(len(shops),
len(storages))

        storages_copy = copy_int_vector(storages)
        shops_copy = copy_int_vector(shops)
        cost_matrix_copy = copy_int_matrix(cost_matrix)

        plan = TransferringElementToPlan(
            sparse_matrix, cost_matrix_copy, transportation_plan,
storages_copy, shops_copy, zero_cors
        )
        plan.calculate()

        cheapest_cors = find_cheapest(plan.cost_matrix)
        plan.set_new_item_cors(cheapest_cors)
        plan.calculate()

    while zero_cors and plan.storages and plan.shops and
plan.sparse_matrix:
        zero_cors = find_only_zero_items(plan.sparse_matrix,
```

```

plan.cost_matrix, plan.ignoring_rows, plan.ignoring_columns)
    if not zero_cors:
        break

    plan.set_new_item_cors(zero_cors)
    plan.calculate()

result_summ = sum(
    [
        plan.transportation_plan[row_index][column_index] *
cost_matrix[row_index][column_index] for row_index in
        range(len(plan.transportation_plan)) for column_index
in range(len(plan.transportation_plan[row_index]))
    ]
)

return plan.transportation_plan, result_summ

```

Листинг 2 - matrix_dilution

```

def matrix_dilution(cost_matrix: list[list[int]]) ->
list[list[int]]:
    sparse_matrix = []

    for row in cost_matrix:
        min_unit = min(row)
        sparse_matrix.append([item - min_unit for item in row])

    for column_index in range(len(sparse_matrix[0])):
        min_unit = min([item[column_index] for item in
sparse_matrix])
        for item in sparse_matrix:
            item[column_index] -= min_unit

    return sparse_matrix

```

Листинг 3 - find_only_zero_items


```

def find_only_zero_items(
    sparse_matrix: list[list[int]],
    cost_matrix: list[list[int]],
    ignoring_rows: list[int] = (),
    ignoring_columns: list[int] = ()
) -> list[int]:
    zero_items_cors = []

    for row_index in range(len(sparse_matrix)):
        checking_row = replace_item_by_index(sparse_matrix[row_index], ignoring_columns)
        if checking_row.count(0) == 1 and row_index not in ignoring_rows:
            zero_items_cors.append([row_index,
sparse_matrix[row_index].index(0)])

    for column_index in range(len(sparse_matrix[0])):
        column = replace_item_by_index([item[column_index] for
item in sparse_matrix], ignoring_rows)
        if column.count(0) == 1 and column_index not in ignoring_columns:
            zero_items_cors.append([column.index(0), column_index])

    if not zero_items_cors:
        return []

    response_zero_item = zero_items_cors[0]

    for zero_item in zero_items_cors:
        if cost_matrix[zero_item[0]][zero_item[1]] <
cost_matrix[response_zero_item[0]][response_zero_item[1]]:
            response_zero_item = zero_item

    return response_zero_item

```

Листинг 4 - replace_item_by_index

```

def replace_item_by_index(vector: list[int],
indexes_of_deleted_items: list[int],
new_value: int) -> list[int]:
NEGATIVE_IMPOSSIBLE_VALUE) -> list[int]:
    for index in indexes_of_deleted_items:
        vector[index] = new_value
    return vector

```

Листинг 5 - paired_transformation

```

def paired_transformation(zero_cors: list[int], sparse_matrix:
list[list[int]], cost_matrix: list[list[int]],
storages: list[int], shops: list[int]):
    resp_sparse_matrix = copy_int_matrix(sparse_matrix)
    resp_sparse_matrix = harmful_action(cost_matrix,
resp_sparse_matrix, zero_cors)
    resp_sparse_matrix = useful_action(cost_matrix,
resp_sparse_matrix, zero_cors)
    delta = delta_calculation(cost_matrix, storages, shops,
zero_cors)

    if delta == 0:
        return resp_sparse_matrix

    if delta > 0:
        resp_sparse_matrix = paired_transformation(zero_cors,
resp_sparse_matrix, cost_matrix, storages, shops)
        return resp_sparse_matrix

    return sparse_matrix

```

Листинг 6 - copy_int_matrix

```

def copy_int_matrix(matrix: list[list[int]]) ->
list[list[int]]:
    return [row[:] for row in matrix]

```

Листинг 7 - harmful_action

```

def harmful_action(cost_matrix: list[list[int]],
sparse_matrix: list[list[int]], item_cors: list[int]):

```

```

cost_item = cost_matrix[item_cors[0]][item_cors[1]]

for item_index in range(len(sparse_matrix[item_cors[0]])):
    sparse_matrix[item_cors[0]][item_index] += cost_item

return sparse_matrix

```

Листинг 8 - useful_action

```

def useful_action(cost_matrix: list[list[int]],
                  sparse_matrix: list[list[int]], item_cors:
list[int]):
    cost_item = cost_matrix[item_cors[0]][item_cors[1]]

    for item_index in range(len(sparse_matrix)):
        sparse_matrix[item_index][item_cors[1]] -= cost_item

    return sparse_matrix

```

Листинг 9 - delta_calculation

```

def delta_calculation(cost_matrix: list[list[int]], storages:
list[int], shops: list[int], item_cors: list[int]):
    item_value = cost_matrix[item_cors[0]][item_cors[1]]
    supply_vector = create_zero_vector(len(storages), item_cors[0],
item_value)
    shop_vector = create_zero_vector(len(shops), item_cors[1],
item_value)
    delta = vectors_multiplication(shop_vector, shops) -
vectors_multiplication(supply_vector, storages)
    return delta

```

Листинг 10 - create_zero_vector

```

def create_zero_vector(len_vector: int, item_position: int,
item_value: int) -> list[int]:
    vector = [0 for _ in range(len_vector)]
    vector[item_position] = item_value
    return vector

```

Листинг 11 - vectors_multiplication

```

def vectors_multiplication(first_vector: list[int],
second_vector: list[int]) -> int:
    return sum(x * y for x, y in zip(first_vector, second_vector))

```

Листинг 12 - create_zero_matrix

```

def create_zero_matrix(rows: int, columns: int) ->
list[list[int]]:
    return [[0 for _ in range(rows)] for _ in range(columns)]

```

Листинг 13 - copy_int_vector

```

def copy_int_vector(vector: list[int]) -> list[int]:
    return [item for item in vector]

```

Листинг 14 - find_cheapest

```

def find_cheapest(matrix: list[list[int]], ignoring_rows:
list[int] = (),
                    ignoring_columns: list[int] = ()) -> list[int]:
    cheapest = IMPOSSIBLE_CORS

    for row_index in range(len(matrix)):
        checking_row = replace_item_by_index(matrix[row_index],
        ignoring_rows, IMPOSSIBLE_VALUE)
        for item_index in range(len(checking_row)):
            if checking_row[item_index] <
matrix[cheapest[0]][cheapest[1]] and item_index not in
ignoring_columns:
                cheapest = [row_index, item_index]

    return cheapest

```

Листинг 15 - TransferringElementToPlan

```

class TransferringElementToPlan:
    def __init__(
        self, sparse_matrix: list[list[int]],
        cost_matrix: list[list[int]],
        transportation_plan: list[list[int]],
        storages: list[int],
        shops: list[int],
    ):

```

```

        item_cors: list[int]
    ):
        self.sparse_matrix = sparse_matrix
        self.cost_matrix = cost_matrix
        self.transportation_plan = transportation_plan
        self.storages = storages
        self.shops = shops
        self.item_cors = item_cors
        self.storages_item = storages[item_cors[FIRST_INDEX]]
        self.shops_item = shops[item_cors[SECOND_INDEX]]
        self.ignoring_rows = []
        self.ignoring_columns = []

    def set_new_item_cors(self, item_cors: list[int]):
        self.item_cors = item_cors
        self.storages_item =
self.storages[item_cors[FIRST_INDEX]]
        self.shops_item = self.shops[item_cors[SECOND_INDEX]]

    def calculate(self):
        self.comparing_items()

    def comparing_items(self):
        if self.shops_item > self.storages_item:
            self.transfer_storage_to_plan()
        elif self.shops_item == self.storages_item:
            self.transfer_shop_and_storage_to_plan()
        else:
            self.transfer_shop_to_plan()

    def transfer_storage_to_plan(self):

self.transportation_plan[self.item_cors[FIRST_INDEX]][self.item_co
rs[SECOND_INDEX]] = self.storages_item
        self.shops[self.item_cors[SECOND_INDEX]]

```

```

self.storages_item

        self.ignoring_rows.append(self.item_cors[FIRST_INDEX])

    def transfer_shop_and_storage_to_plan(self):

self.transportation_plan[self.item_cors[FIRST_INDEX]][self.item_co
rs[SECOND_INDEX]] = self.storages_item

self.ignoring_columns.append(self.item_cors[SECOND_INDEX])
        self.ignoring_rows.append(self.item_cors[FIRST_INDEX])

    def transfer_shop_to_plan(self):

self.transportation_plan[self.item_cors[FIRST_INDEX]][self.item_co
rs[SECOND_INDEX]] = self.shops_item
        self.storages[self.item_cors[FIRST_INDEX]] -=
self.shops_item

self.ignoring_columns.append(self.item_cors[SECOND_INDEX])

```